AI-K8sGuard: System Architecture, Design, Methodology and Implementation

An Explainable AI-Assisted Framework for Kubernetes Security Assessment and Failure Diagnosis Using a Hybrid Rule-Based and Local LLM Approach

Abstract— This document presents the architecture, design, methodology and implementation of AI-K8sGuard, a lightweight, local-first framework that combines deterministic Kubernetes security analysis with a locally hosted large language model (LLM). Kubernetes Deployment YAML files and selected live-cluster resources are analyzed by a rule engine that produces severity-rated findings and a transparent risk score. Relevant configuration, status, event and log information is then organized into structured evidence and supplied to a Qwen2.5 3B model served through Ollama, which produces explanations, probable causes, impact descriptions and recommendations. The document is organized in two parts: Part I covers system architecture and design (architecture, module decomposition, workflow, data flow, technology stack, design decisions and module interfaces); Part II covers methodology and implementation (algorithms, models, frameworks, tools, database design, hardware, APIs, integration procedure and experimental evaluation).

Index Terms— Kubernetes security, security misconfiguration, failure diagnosis, explainable AI, local LLM, evidence grounding, risk assessment.

Note on content. The architecture, pipeline, rule categories, evidence model, failure scenarios, technology choices and evaluation design come from the original paper. Specific details the paper leaves open (for example endpoint names, severity weights, database field names and library choices) are marked **(proposed)** so they can be confirmed or changed before submission.

PART I. SYSTEM ARCHITECTURE AND DESIGN

1.1 Design Overview and Principles

AI-K8sGuard is designed to assess the security of Kubernetes workloads and to help diagnose operational failures without sending cluster data to a paid external AI service. The central design principle is a strict separation of responsibilities: deterministic rules decide what is a security finding and how serious it is, while the local LLM only interprets verified evidence. This follows from published observations that LLMs can detect and explain Kubernetes misconfigurations [1] but can also produce plausible diagnoses that are not grounded in the telemetry provided to them [2], and that static analysis and LLM reasoning are complementary rather than interchangeable [3].

The architecture is guided by five principles:

- **Determinism for decisions:** the same input configuration always yields the same findings and risk score.
- **Evidence before explanation:** the LLM receives only scoped, structured evidence tied to a verified finding or observed failure.
- **Local-first operation:** all components run on local infrastructure, supporting data locality and reproducibility.
- **Lightweight scope:** the framework targets selected checks and failure scenarios rather than replacing enterprise security platforms or complex diagnosis systems such as MetaKube [4].
- **Advisory AI:** the LLM never modifies cluster resources; recommendations are presented for a human to act on.

1.2 Proposed Architecture

The system receives information from two sources: Kubernetes Deployment YAML files (static analysis) and a live Kubernetes cluster (runtime analysis). The overall processing pipeline is:

YAML/Cluster Input → Analysis → Security Rules → Risk Assessment → Evidence Builder → Local LLM → Explainable Report

Logically, the system is organized into the six layers shown in Fig. 1, from the user interface at the top to the local infrastructure at the bottom.

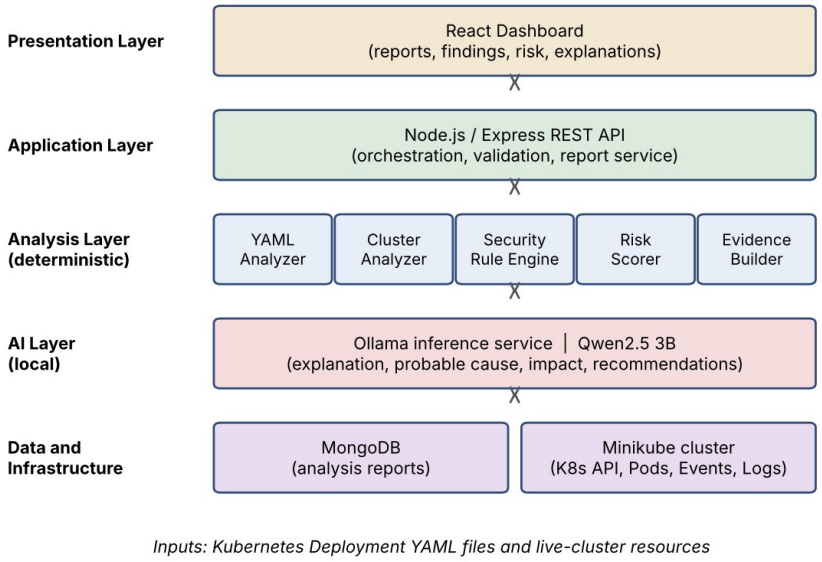


Fig. 1. Layered architecture of the proposed AI-K8sGuard framework.

- **Input and Collection Layer:** receives YAML files and collects selected live-cluster information (Pods, Deployments, Services, Events, status and logs).
- **Deterministic Analysis Layer:** evaluates configurations against predefined security rules and assigns severity.
- **Risk Assessment Layer:** aggregates findings into a transparent severity-weighted score.
- **Evidence Construction Layer:** combines each finding or failure with the relevant configuration and runtime information.
- **Local AI Layer:** Ollama serving Qwen2.5 3B interprets the structured evidence.
- **Reporting Layer:** stores results in MongoDB and presents findings, risk, explanations and diagnoses through the web dashboard.

1.3 Module Decomposition

The framework is decomposed into ten modules with clearly bounded responsibilities (Table I). The decomposition into modules M1–M10 is a proposed implementation view of the layers above.

TABLE I. MODULE DECOMPOSITION

ID	Module	Responsibility	Input → Output
M1	YAML Security Analyzer	Parses Deployment YAML and extracts security context, privilege settings, resource limits, probes, image, host-level settings and capabilities.	YAML text → normalized workload configuration

ID	Module	Responsibility	Input → Output
M2	Kubernetes Cluster Analyzer	Reads Pods, Deployments, Services, Events, status and logs through the Kubernetes API (read-only).	Cluster/namespace selector → runtime snapshot
M3	Failure Scenario Detector (proposed)	Recognizes supported failure states: CrashLoopBackOff, ImagePullBackOff, missing Secrets/ConfigMaps and readiness failures.	Runtime snapshot → failure records
M4	Security Rule Engine	Evaluates predefined rules and generates structured findings F_i with severity and supporting evidence.	Normalized configuration → findings
M5	Risk Assessment Module	Computes the cumulative severity-weighted risk score R.	Findings → risk score
M6	Evidence Builder	Assembles bounded evidence E = F + Y + S + V + L for each finding or failure.	Findings, failures, YAML, status, events, logs → evidence package
M7	Local LLM Service	Builds the prompt, calls Ollama (Qwen2.5 3B) and parses the explanation, cause, impact, severity interpretation and recommendations.	Evidence package → structured explanation
M8	Report Service	Combines findings, risk, evidence and LLM output into a report and stores it.	All module outputs → stored report
M9	Web Dashboard	React interface for uploading YAML, triggering cluster analysis and inspecting reports.	User actions ↔ reports
M10	Evaluation Harness (proposed)	Runs controlled scenarios in rule-only, LLM-only and hybrid modes and compares results with ground truth.	Scenario set → metrics

1.4 System Workflow

The workflow has two entry points that converge on the same evidence and reporting stages.

A. Security Assessment Workflow

1. The user uploads a Deployment YAML file, or selects a namespace/workload in the live cluster, through the dashboard.
2. M1 (static) and/or M2 (live) produce a normalized representation of the workload.
3. M4 evaluates every security rule and emits findings, each with a rule identifier, severity and supporting evidence.
4. M5 aggregates the findings into a risk score.
5. M6 builds an evidence package for each finding, adding relevant YAML, status, events and logs.
6. M7 sends the evidence package to the local LLM and receives an explanation, probable cause, impact and recommendation.
7. M8 stores the complete report in MongoDB and the dashboard displays it.

B. Failure Diagnosis Workflow

1. M2 collects Pod and Deployment status, Events and logs for the affected workload.
2. M3 identifies a supported failure state from the status and events.
3. M6 assembles the failure evidence (status, events, logs and the related configuration).

4. M7 asks the local LLM for probable cause, impact and recommended investigation or remediation actions.
5. M8 stores and presents the diagnosis alongside any security findings for the same workload.

1.5 Data Flow

Data moves in one direction through the pipeline, becoming progressively more structured and more trusted (Fig. 2). Raw configuration and runtime data enter at the left; rule evaluation converts them into verified findings; the Evidence Builder converts findings and runtime context into a bounded evidence package; and the LLM converts the evidence package into a human-readable explanation. Only verified findings and scoped evidence reach the LLM; unrestricted cluster information is never forwarded.

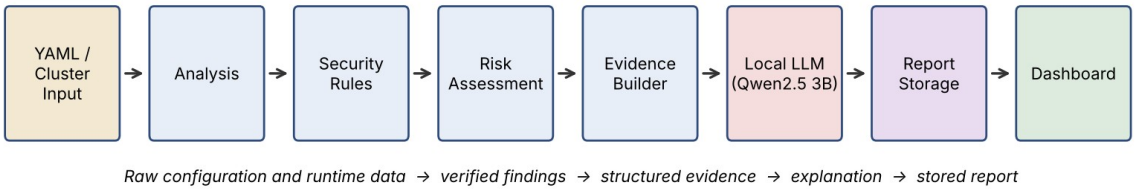


Fig. 2. End-to-end data flow of the AI-K8sGuard pipeline.

The principal data objects are:

- **Finding** $F_i = \{r_i, e_i, s_i, w_i\}$: rule, supporting evidence, severity and risk contribution.
- **Risk score** R: cumulative weight of all findings for a workload.
- **Evidence package** E = F + Y + S + V + L: finding, YAML configuration, workload status, events/runtime information and relevant logs.
- **LLM output:** explanation, probable root cause, potential impact, severity interpretation and recommended actions.
- **Report:** Finding, Severity, Risk, Evidence, Explanation, Cause, Recommendation, stored as one document per analysis.

1.6 Technology Stack

TABLE II. TECHNOLOGY STACK

Layer	Technology	Role
Container orchestration	Kubernetes on Minikube	Local, reproducible cluster supplying workloads, Events, logs and the Kubernetes API
Local LLM runtime	Ollama	Serves the model locally through an HTTP API
Language model	Qwen2.5 3B	Explanation and failure diagnosis over structured evidence
Backend	Node.js with Express	REST API, analyzers, rule engine, evidence builder and LLM client
Frontend	React	Dashboard for uploads, analysis requests and report inspection
Database	MongoDB	Stores analysis reports as documents
Input format	Kubernetes Deployment YAML	Static configuration under analysis

All components are open source or locally deployable, so no paid cloud AI service is required.

1.7 Design Decisions

TABLE III. KEY DESIGN DECISIONS AND RATIONALE

Decision	Rationale and trade-off
Rules decide security findings; the LLM only explains them	Gives reproducible results and prevents an unsupported model output from changing a security conclusion. Trade-off: coverage is limited to implemented rules.
Structured, scoped evidence instead of raw cluster data	Makes the link between an explanation and its source explicit, supports grounding evaluation [2] and fits the limited context of a small model.
Local LLM through Ollama	Keeps configuration and operational data on local infrastructure, avoids paid APIs and supports reproducible experiments.
Small model (Qwen2.5 3B)	Runs on modest hardware. Trade-off: lower reasoning capability than larger models, which is why larger models are listed as future work.
Static and live analysis combined	Distinguishes what a workload specifies from its observed state in the cluster.
Severity-weighted additive risk score	Simple and transparent for prioritization and comparison; it is not a universal security measure.
Document database (MongoDB)	A report contains nested, variable-length data (findings, evidence, LLM output), which maps naturally onto documents.
Read-only cluster access and no automatic remediation	Limits risk; any future remediation must require explicit validation and user approval.
Minikube as the test environment	Provides a self-contained, repeatable cluster for controlled security and failure scenarios.

1.8 Interfaces Between Modules

Modules communicate through well-defined data contracts. Because the backend is a single Node.js application, interfaces between analysis modules are in-process function calls exchanging JSON objects; the dashboard, Kubernetes, Ollama and MongoDB interfaces are network calls (Table IV).

TABLE IV. MODULE INTERFACES

ID	From → To	Mechanism	Content
I1	Dashboard → Backend API	HTTP REST (JSON / file upload)	YAML file, analysis and report requests
I2	Backend API → M1 / M2	In-process call	YAML text; namespace and workload selector
I3	M2 → Kubernetes API	HTTPS REST via client library (read-only)	Pods, Deployments, Services, Events, logs
I4	M1 / M2 → M4 (and M3)	In-process JSON object	Normalized configuration and runtime snapshot
I5	M4 → M5	In-process JSON array	Findings with severity and weight
I6	M4 / M3 / M1 / M2 → M6	In-process JSON object	Findings, failures, YAML, status, events, logs
I7	M6 → M7 → Ollama	HTTP to local Ollama endpoint	Prompt containing the evidence package; model response
I8	M7 → M8	In-process JSON object	Explanation, cause, impact, severity interpretation, recommendations

ID	From → To	Mechanism	Content
I9	M8 ↔ MongoDB	Database driver	Report documents
I10	Backend API → Dashboard	HTTP REST (JSON)	Stored reports for display

PART II. METHODOLOGY AND IMPLEMENTATION

2.1 Algorithms

A. Deterministic Rule Evaluation

The Security Rule Engine applies each predefined rule to the normalized workload configuration and emits a finding when the condition holds. The supported rule categories are privileged containers, root execution, privilege escalation, missing resource limits, missing readiness or liveness probes, mutable image tags, host-level configuration risks and excessive capabilities. Table V lists the checks and an initial severity assignment (proposed).

TABLE V. SECURITY RULES AND INITIAL SEVERITY (PROPOSED)

Rule	Condition checked	Typical field	Severity
R1 Privileged container	Container runs in privileged mode	securityContext.privileged	Critical
R2 Root execution	Runs as UID 0 or does not require non-root	runAsUser, runAsNonRoot	High
R3 Privilege escalation	Privilege escalation permitted or not disabled	allowPrivilegeEscalation	High
R4 Host-level access	Host network, PID, IPC or host path used	hostNetwork, hostPID, hostIPC, hostPath	High
R5 Excessive capabilities	Dangerous capabilities added	capabilities.add	High
R6 Missing resource limits	CPU/memory limits not set	resources.limits	Medium
R7 Mutable image tag	Image uses latest or has no explicit tag	image	Medium
R8 Missing probes	No readiness or liveness probe	readinessProbe, livenessProbe	Low

A finding is represented as:

$$F_i = \{ r_i, e_i, s_i, w_i \}$$

where r_i is the security rule, e_i the supporting evidence, s_i the assigned severity and w_i the risk contribution. Rule evaluation is deterministic: the same input always produces the same set of findings.

```
Algorithm 1: Rule evaluation
Input : normalized workload configuration W, rule set RULES
Output: findings F
F <- empty list
for each rule r in RULES:
    for each container c in W:
        if r.condition(c) is true:
            e <- extract_field_evidence(c, r.field)
            F.append({ rule: r.id, evidence: e,
                       severity: r.severity,
                       weight: WEIGHT[r.severity] })
return F
```

B. Severity-Weighted Risk Scoring

Findings are aggregated into a cumulative risk value so that more serious findings contribute more to the overall score:

$$R = \Sigma\, w_i \quad (i = 1 \ldots n)$$

where n is the number of findings and w_i is the contribution of finding i. As initial values (proposed), the weights are Critical = 10, High = 7, Medium = 4 and Low = 2; these can be calibrated during evaluation. The score is intended for prioritization and consistent comparison of controlled configurations, not as a universal measure of Kubernetes security.

C. Evidence Construction

After a finding or operational failure is identified, the Evidence Builder gathers only the information associated with that condition:

$$E = F + Y + S + V + L$$

where F is the verified finding, Y the relevant YAML/configuration evidence, S the workload status (Pod and Deployment), V the events and related runtime information, and L the relevant logs. Bounding the evidence keeps the prompt within the limited context of a small model and makes each explanation traceable to specific inputs.

```
Algorithm 2: Evidence construction and LLM interpretation
Input : finding or failure f, YAML Y, runtime snapshot Rt
Output: explanation X
S <- status_of(f.workload, Rt)
V <- events_for(f.workload, Rt)
L <- logs_for(f.workload, Rt)  (bounded)
E <- { finding: f, yaml: select(Y, f), status: S,
       events: V, logs: L }
prompt <- TEMPLATE(E)
raw <- ollama.generate(model, prompt)
X <- parse_structured(raw)
return X
```

D. Failure Scenario Detection

The framework supports selected, commonly encountered failure scenarios. Each is recognized from observable Kubernetes state and then diagnosed with the corresponding evidence (Table VI).

TABLE VI. SUPPORTED FAILURE SCENARIOS AND EVIDENCE

Scenario	Observable indicators used as evidence
CrashLoopBackOff	Container waiting reason, restart count, container logs, related Events
ImagePullBackOff	Image-pull waiting reason, pull-failure Events, image reference in the YAML
Missing Secret or ConfigMap	Container configuration error status, Events naming the missing object, references in the YAML
Readiness failure	Pod Ready condition, probe-failure Events, probe configuration in the YAML

2.2 Models and Frameworks

Language model. Explanation and diagnosis use Qwen2.5 3B, a small open model that can run locally on modest hardware. **Inference framework.** Ollama hosts the model and exposes it through a local HTTP API, so no external service is called.

Prompt and output design (proposed). The prompt contains (i) an instruction to base the answer only on the supplied evidence and to state when evidence is insufficient, (ii) the evidence package, and (iii) the required output fields. A low sampling temperature is used to reduce variability between runs, and the output is requested in a structured form with the fields: explanation, probable cause, potential impact, severity interpretation and recommended actions. The model is not asked to decide whether a configuration is secure; the verified finding is supplied to it as input.

Architectural framework. The application follows a layered client–server design: a React single-page front end, a Node.js/Express REST back end, a MongoDB data store, and Ollama and Kubernetes as external local services.

2.3 Development Tools and Programming Languages

TABLE VII. DEVELOPMENT TOOLS

Category	Tool / language	Purpose
Languages	JavaScript (Node.js, React); YAML for workloads	Back end, front end and test manifests
Back-end framework	Express	REST API
Front-end framework	React	Dashboard
YAML parsing (proposed)	A Node.js YAML library	Convert YAML to objects for M1
Kubernetes client (proposed)	Official Kubernetes client for JavaScript	Read-only API access for M2
Database driver (proposed)	MongoDB driver / ODM	Report persistence
Cluster tooling	Minikube, kubectl	Create and inspect test workloads
Model tooling	Ollama CLI	Download and serve Qwen2.5 3B
Version control	Git	Source management

2.4 Database Design

MongoDB stores one report document per analysis run. The report fields follow the output specified by the framework (Finding, Severity, Risk, Evidence, Explanation, Cause, Recommendation); the field names in Table VIII are proposed.

TABLE VIII. REPORT DOCUMENT STRUCTURE (PROPOSED)

Field	Type	Description
_id, createdAt	ObjectId, date	Identifier and creation time
source	string	yaml or cluster
target	object	Workload name, namespace and kind
mode	string	rule-only, llm-only or hybrid (for experiments)
yamlSnapshot	string	Analyzed configuration
findings[]	array of objects	ruleId, severity, weight and evidence for each finding
riskScore	number	Cumulative risk R
evidence	object	Status, events, logs and configuration used for interpretation
failure	object	Detected failure scenario, if any
llm	object	model, explanation, cause, impact, severityInterpretation, recommendations[], latencyMs

Embedding findings and LLM output inside the report document means a complete report can be retrieved with a single query. An optional second collection can hold evaluation runs and metrics (proposed).

2.5 Hardware Components

The prototype is designed to run on a single local machine (workstation or laptop) that hosts Minikube, MongoDB, the Node.js back end, the React front end and the Ollama service. Qwen2.5 3B is small enough for local inference on commodity hardware, with or without a GPU. The exact processor, memory, GPU and storage configuration is not fixed by the design; it will be recorded together with the latency and resource-usage measurements so that results are reproducible.

2.6 Application Programming Interfaces

Three API boundaries are involved. The back-end REST API serves the dashboard (endpoint names are proposed); the Kubernetes API is used read-only; the Ollama API provides inference.

TABLE IX. APIs USED BY THE FRAMEWORK

API	Operation	Purpose
Backend: POST /api/analyze/yaml (proposed)	Upload YAML	Static security analysis and report
Backend: POST /api/analyze/cluster (proposed)	Namespace/workload selector	Live analysis and report
Backend: POST /api/diagnose (proposed)	Workload selector	Failure diagnosis
Backend: GET /api/reports, GET /api/reports/:id (proposed)	List / fetch	Retrieve stored reports
Kubernetes API (read-only)	List/read Pods, Deployments, Services, Events; read Pod logs	Runtime evidence
Ollama HTTP API	Generate/chat request to the qwen2.5:3b model	Local LLM inference

2.7 Integration Procedure

The components are integrated and verified in the following order:

1. Start the local Kubernetes cluster with Minikube and confirm access with kubectl.
2. Deploy the controlled test workloads: secure baselines, deliberately misconfigured Deployments and Deployments that trigger the supported failure scenarios.
3. Start MongoDB and confirm the back end can connect.
4. Install Ollama, download the model (ollama pull qwen2.5:3b) and confirm that a test prompt returns a response.
5. Configure and start the Express back end with the cluster access configuration, the Ollama endpoint and the database address.
6. Start the React dashboard and connect it to the back end.
7. Run a YAML analysis and confirm that findings and a risk score are returned deterministically for repeated runs.
8. Run a live-cluster analysis and a failure diagnosis, and confirm that evidence is collected and the LLM output is parsed.
9. Confirm that the report is stored in MongoDB and displayed on the dashboard.
10. Run the evaluation harness across the three configurations.

2.8 Experimental Methodology

The evaluation compares three configurations using controlled Kubernetes security and failure scenarios:

1. **Rule-only:** only deterministic rules identify and score security findings.
2. **LLM-only:** the local LLM receives the relevant Kubernetes information without the deterministic decision layer.
3. **Hybrid:** rules establish the findings, then structured evidence is given to the local LLM for explanation and diagnosis.

The experimental workflow is:

Controlled Scenario → Rule-only / LLM-only / Hybrid → Generated Result → Ground-Truth Comparison → Performance Evaluation

TABLE X. EVALUATION CRITERIA

Criterion	What is measured
Security-detection accuracy	Agreement of detected findings with the known ground truth of each scenario
Failure-diagnosis accuracy	Whether the identified cause matches the injected failure
Evidence grounding	Whether claims in the explanation are supported by the evidence supplied (scoring rubric proposed)
Explainability	Clarity and usefulness of the explanation, cause and recommendation (rubric proposed)
Latency	Time from request to completed report
Computational requirements	CPU and memory use during local inference

The purpose is to determine whether combining deterministic analysis with local LLM interpretation gives measurable benefits, without assuming that the LLM is inherently more accurate than rule-based analysis.

2.9 Summary

AI-K8sGuard separates deterministic security decisions from local-LLM interpretation, passing only verified findings and scoped Kubernetes evidence to Qwen2.5 3B. The layered, modular design runs entirely on local infrastructure using Minikube, Ollama, Node.js/Express, React and MongoDB, and is evaluated through controlled rule-only, LLM-only and hybrid experiments. Its aim is to assess the practical value and limits of explainable, evidence-grounded, local LLM assistance for Kubernetes security assessment and failure diagnosis.

REFERENCES

[1] E. Malul, Y. Meidan, D. Mimran, Y. Elovici, and A. Shabtai, “GenKubeSec: LLM-based Kubernetes misconfiguration detection, localization, reasoning, and remediation,” arXiv preprint arXiv:2405.19954, 2024.

[2] R. Patil and S. Bhagwat, “Confident liars: Evidence-grounded root cause analysis benchmarking for Kubernetes incident response,” unpublished manuscript, 2026.

[3] N. Yang, X. Liu, W. Shen, J. Li, and K. Lu, “Dangers behind access control: Understanding and exploiting implicit permissions in Kubernetes,” in Proc. 2025 ACM SIGSAC Conf. Computer and Communications Security (CCS ’25), Taipei, Taiwan, Oct. 13–17, 2025, pp. 3401–3415, doi: 10.1145/3719027.3765106.

[4] W. Sun, T. Wang, X. Tian, W. Lan, X. Feng, H. Li, and F. Wang, “MetaKube: An experience-aware LLM framework for Kubernetes failure diagnosis,” in Proc. ACM Web Conf. 2026 (WWW ’26), Dubai, United Arab Emirates, Apr. 13–17, 2026, pp. 5612–5623, doi: 10.1145/3774904.3792631.